Spring MVC 기초

학습목표

- Spring Boot 프로젝트를 생성한다.
- Spring Controller와 View를 구현한다.
- Spring Web MVC 실행 절차를 이해한다.

목차

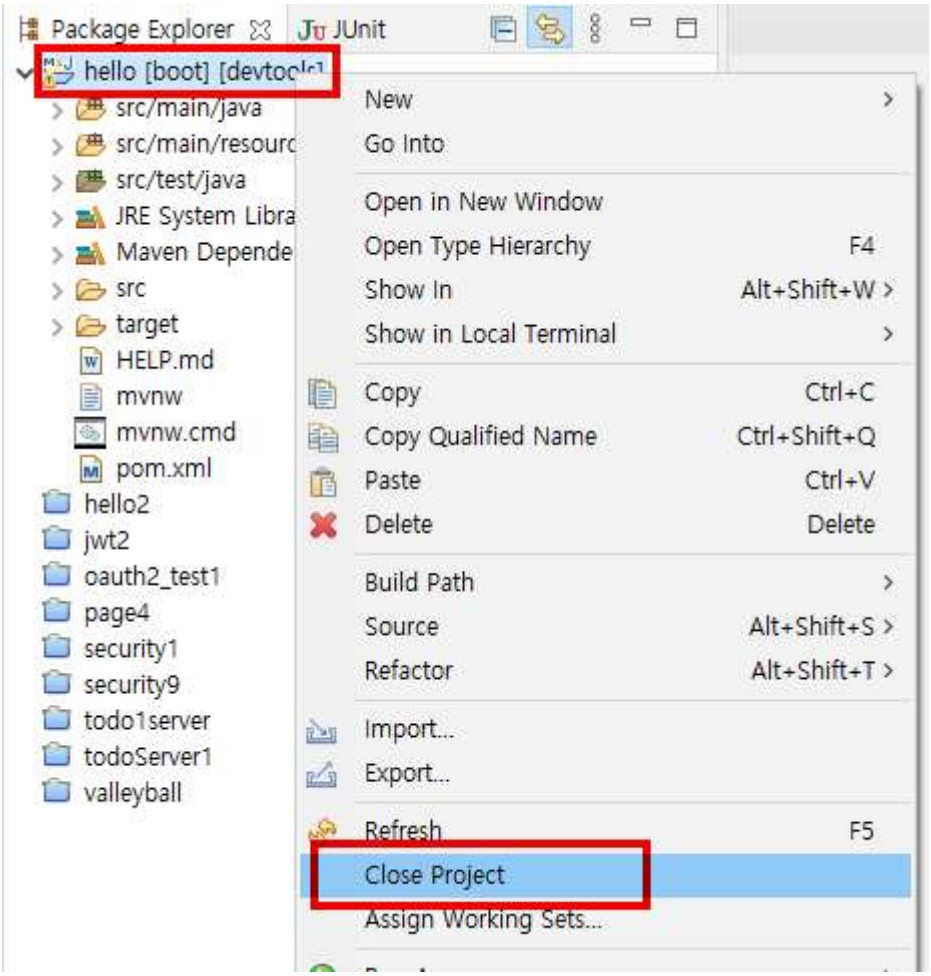
1. 배경 지식 2
    1) 프로젝트 닫기 2
    2) Java 객체의 속성 3
2. hello2 프로젝트 4
    1) 프로젝트 생성 4
    2) application.properties 수정 6
3. FirstController 7
    1) Product.java 생성 7
    2) getter setter 메소드 자동 구현 9
    3) FirstController.java 생성 13
    4) 실행 15
    5) FirstController 액션 메소드 실행 절차 17
    6) JSON 형태의 텍스트 17
4. SecondController 18
    1) 뷰(view) 18
    2) 실행 순서 18
    3) second/test1.html 뷰 파일 생성 19
    4) second/test2.html 뷰 파일 생성 20
    5) SecondController.java 생성 21
    6) 실행 23
    7) 실행 절차 26
5. HomeController 27
    1) HomeController.java 생성 27
    2) home/index.html 생성 28
    3) 실행 29
6. 구현 과제 30
7. 요약 31

1. 배경 지식

1) 프로젝트 닫기

현재 구현 중인 프로젝트만 열어 놓고, 나머지 프로젝트들은 닫으면,
사소한 차이이지만 eclipse가 좀 더 빠르게 실행된다.

hello 프로젝트 닫기



닫은 프로젝트를 다시 열려면, 프로젝트 이름을 더블 클릭하자.

2) Java 객체의 속성

Java 객체의 속성은 멤버 변수가 아니고, get/set 메소드로 구현된다.

```
public class Product {
    public String getUnitCost() { ... }
    public void setUnitCost(int unitCose) { ... }

    public String getName() { ... }
    public void setName(String name) { ... }
}
```

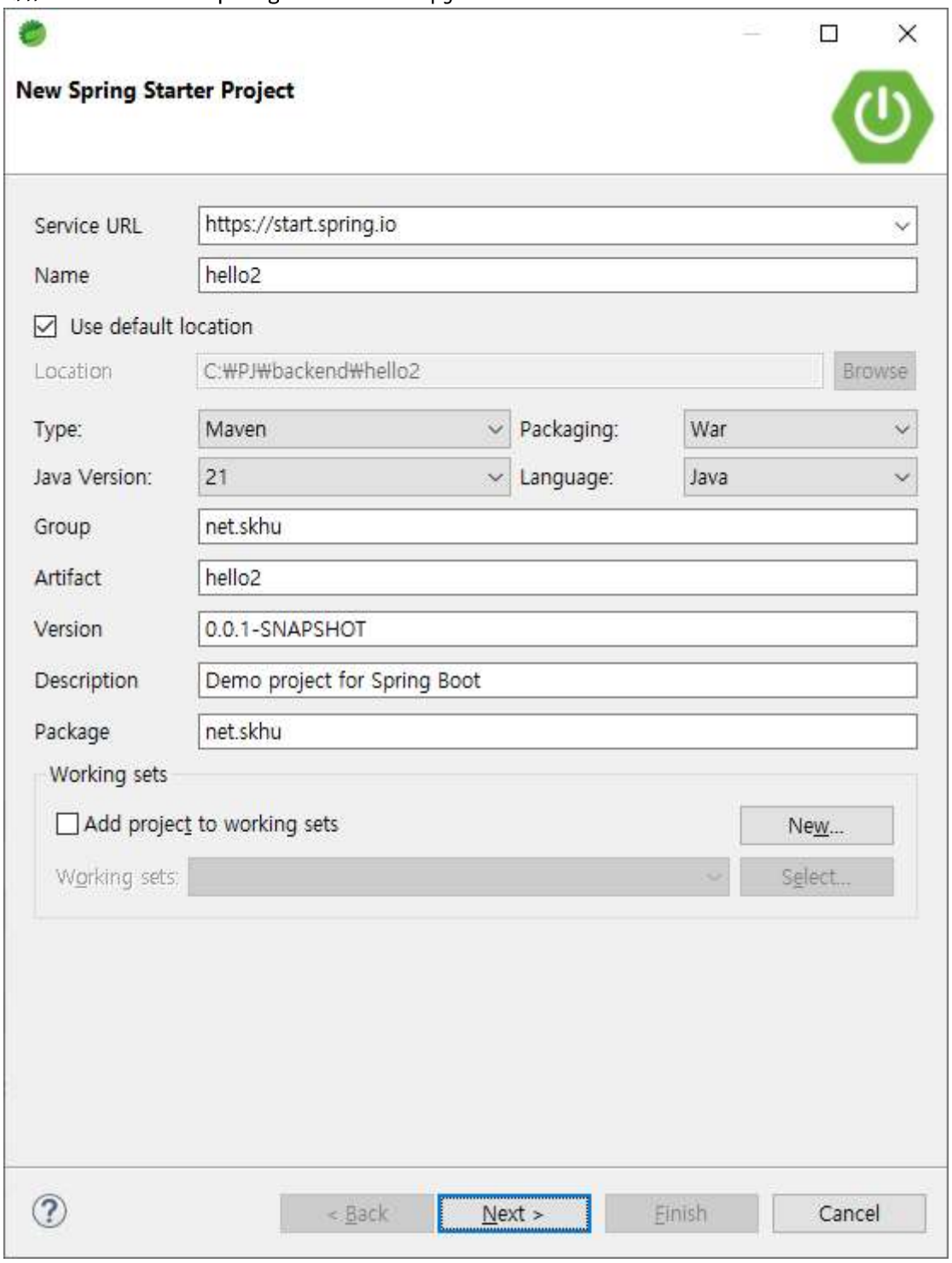
즉 위 Product 클래스에 unitCose, name 두 속성이 있다.

스프링 엔진이나 뷰 템플릿 엔진이
   Java 객체로부터 속성값을 꺼낼 때, get 메소드를 호출한다. 예: getUnitCost(), getName()
   Java 객체에 속성값을 채울 때, set 메소드를 호출한다. 예: setUnitCost(...), setName(...)

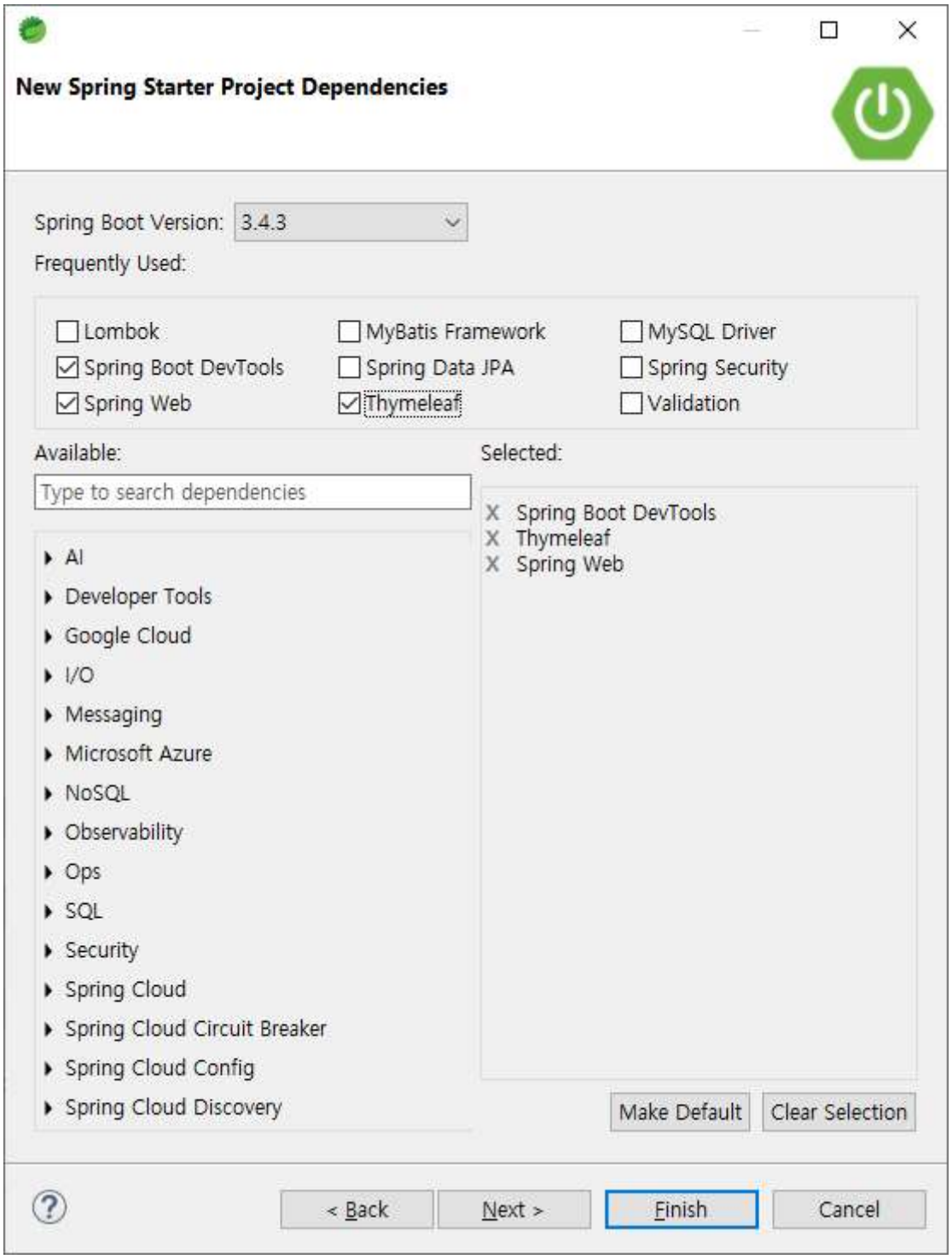
2. hello2 프로젝트

1) 프로젝트 생성

메뉴: File - New Spring Starter Propject



Name	hello2
Type	maven
Packaging	war
Java Version	21
Package	net.skhu



이 예제에서는 데이터베이스 연결을 하지 않을 것이기 때문에,
데이터베이스 항목들을 체크하지 않아야 한다.

Spring Web 항목
   Spring Web MVC 라이브러리

Spring Boot DevTools 항목
   프로젝트를 실행할 때, 프로젝트를 재시작하지 않아도
   수정된 소스 코드가 실행에 즉시 반영될 수 있도록 해주는 개발 도구

Thymeleaf
   뷰(view) 템플릿 엔진

2) application.properties 수정

src/main/resources/application.properties

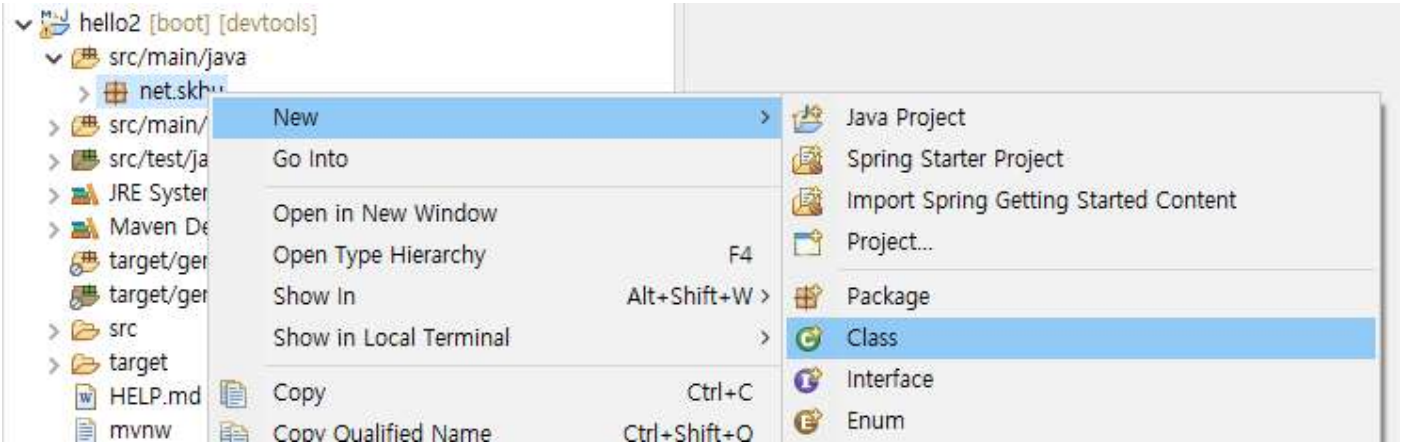
```
spring.application.name=hello2
server.port=8088
```

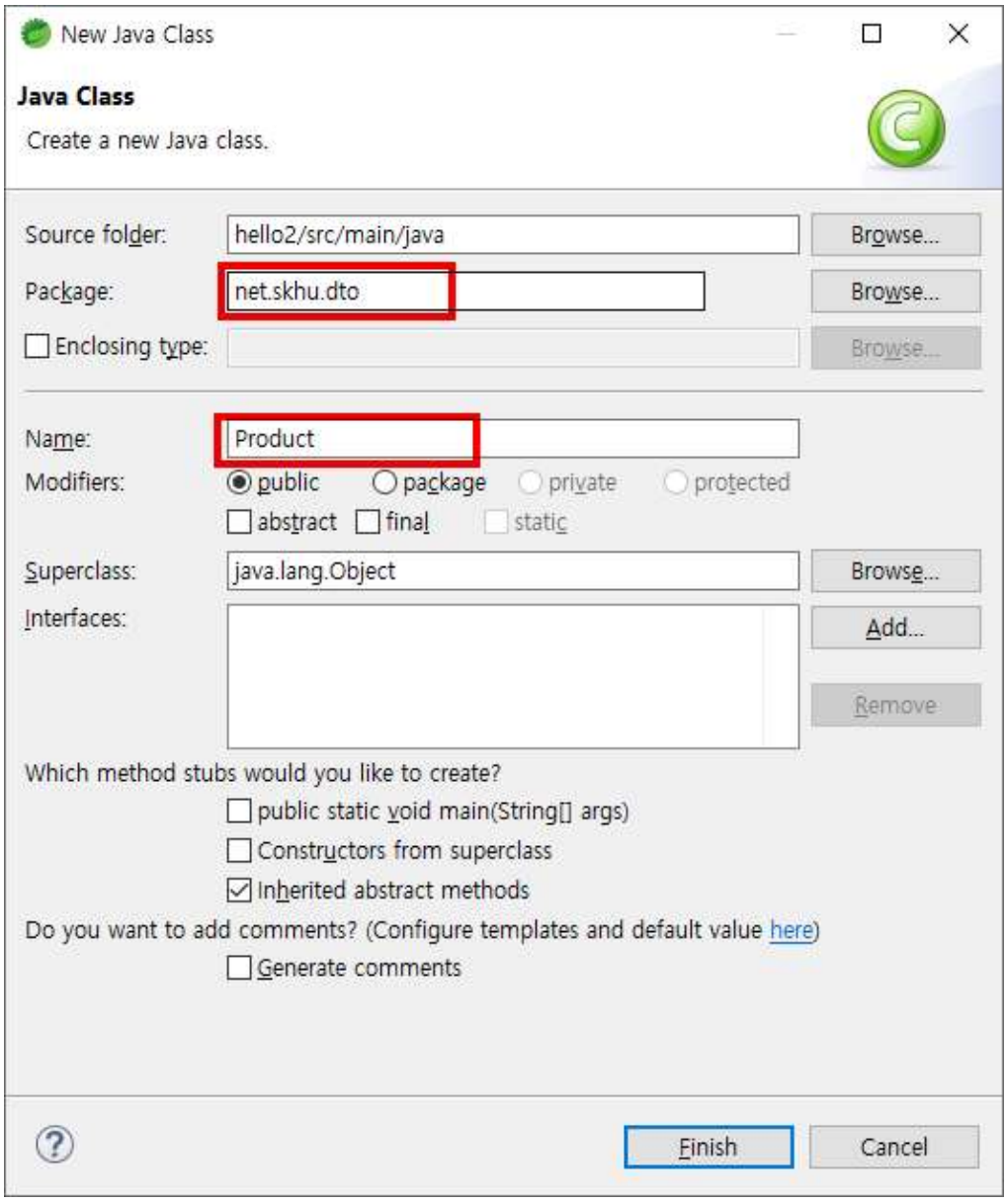
스프링 부트의 설정 파일

내장된 톰캣 서버가 실행될 포트 디폴트 포트는 8080인데,
8080 포트를 사용하는 다른 서버와 충돌이 발생할 확률이 높다.
8088 포트로 변경하자.
   server.port=8088   -> 톰캣 서버가 사용할 포트 번호

3. FirstController

1) Product.java 생성





Package	net.skhu.dto
Name	Product

src/main/java/net/skhu/dto/Product.java

```java
package net.skhu.dto;

public class Product {
    String name;
    int unitCost;

    public Product(String name, int unitCost) {
        this.name = name;
        this.unitCost = unitCost;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getUnitCost() {
        return unitCost;
    }

    public void setUnitCost(int unitCost) {
        this.unitCost = unitCost;
    }

}
```

제품(product)의 이름(name)과 가격(cost) 데이터를 넣어서 전달하기 위한 클래스이다.

데이터를 채워서 전달하기 위한 목적의 클래스를 DTO(Data Transfer Object) 라고 부른다.

2) getter setter 메소드 자동 구현

DTO 클래스를 구현할 때, getter setter 메소드를 손으로 구현하는 것은 매우 지루할 뿐만 아니라 오타 실수를 범할 확률이 높다.
getter setter를 자동 구현하자.

① 먼저 멤버 변수만 구현

```
package net.skhu.dto;

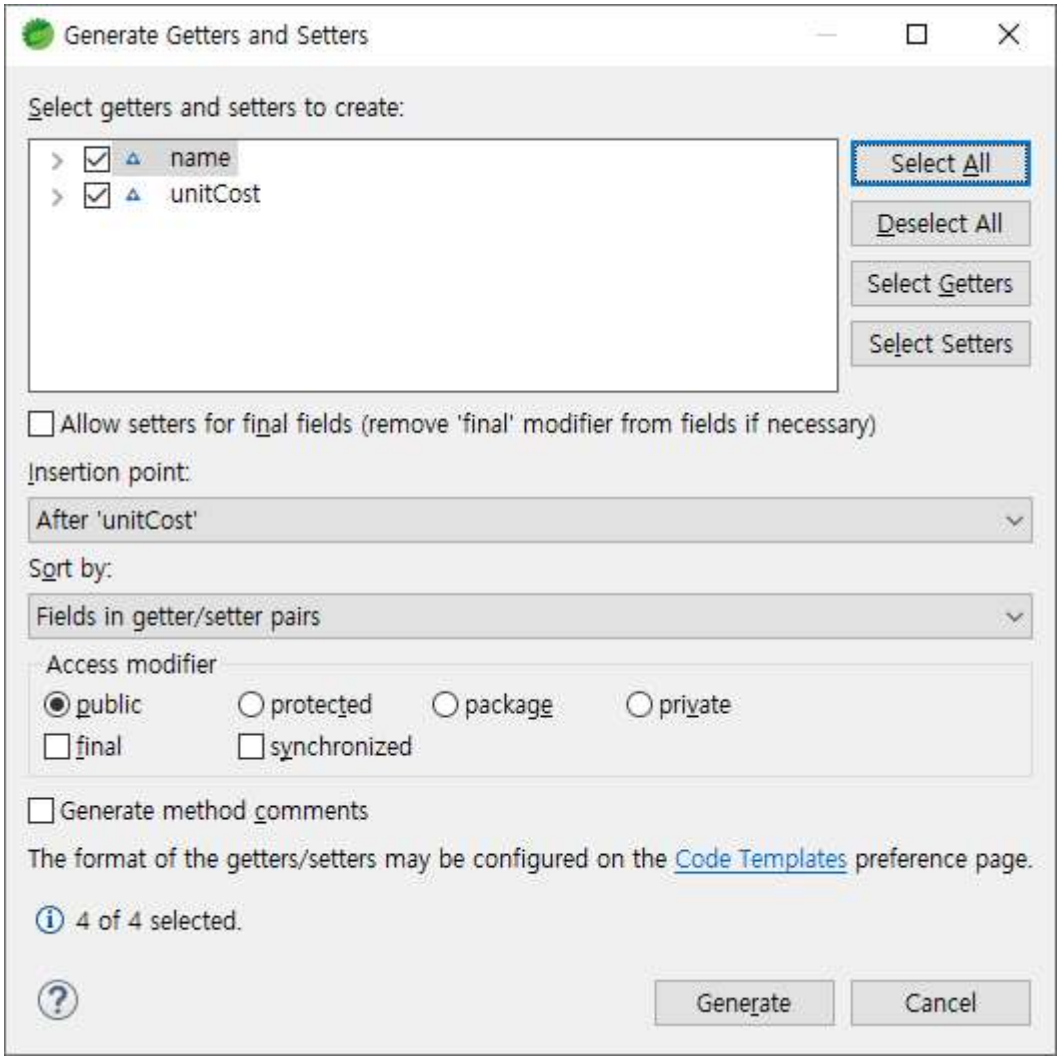
public class Product {
    String name;
    int unitCost;

}
```

멤버 변수만 오타 없이 꼼꼼하게 구현하자.

② getter setter 자동 구현

마지막 멤버 변수 아래에 커서를 위치하고
Source - Generate Getters and Setters 메뉴 클릭



Select All 버튼을 클릭하여, 모든 멤버 변수 선택
Generate 버튼을 클릭

```java
package net.skhu.dto;

public class Product {
    String name;
    int unitCost;
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public int getUnitCost() {
        return unitCost;
    }
    public void setUnitCost(int unitCost) {
        this.unitCost = unitCost;
    }
}
```

getter setter 메소드들이 자동 구현되었다.
그런데 메소드들 사이에 빈줄이 없다.

③ Source Format

Source - Fomrat 메뉴 클릭
단축키 Ctrl+Shift+F

이 메뉴를 클릭하면 소스코드가 자동으로 줄맞춤 된다.

```java
package net.skhu.dto;

public class Product {
    String name;
    int unitCost;

    public String getName() {
        return name;
    }

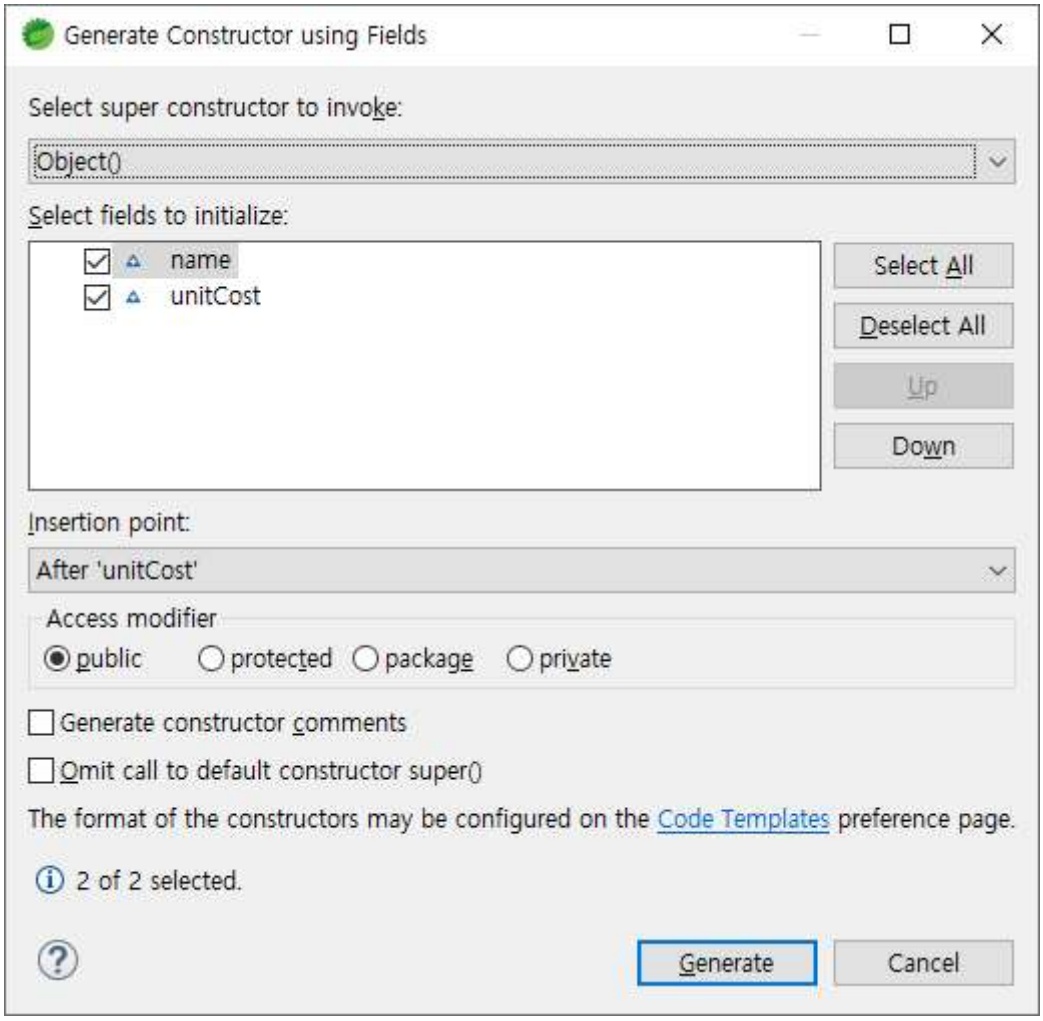
    public void setName(String name) {
        this.name = name;
    }

    public int getUnitCost() {
        return unitCost;
    }

    public void setUnitCost(int unitCost) {
        this.unitCost = unitCost;
    }
}
```

④ 생성자 자동 구현

마지막 멤버 변수 아래에 커서를 위치하고
Source - Generate Constructor using Fields 메뉴 클릭



생성자 파라미터로 전달될 멤버 변수를 선택하고
Generate 버튼 클릭

```java
package net.skhu.dto;

public class Product {
    String name;
    int unitCost;

    public Product(String name, int unitCost) {
        super();
        this.name = name;
        this.unitCost = unitCost;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getUnitCost() {
        return unitCost;
    }

    public void setUnitCost(int unitCost) {
        this.unitCost = unitCost;
    }
}
```

생성자가 자동 구현되었다.

부모 클래스가 없으므로, super(); 호출은 불필요하다. 삭제하자.
(삭제하지 않고 그냥 놔둬도 된다)

3) FirstController.java 생성

src/main/java/net/skhu/controller/FirstController.java

```java
package net.skhu.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import net.skhu.dto.Product;


@RestController
@RequestMapping("first")
public class FirstController {

    @GetMapping("test1")
    public String test1() {
        return "안녕하세요";
    }

    @GetMapping("test2")
    public String[] test2() {
        return new String[] { "월", "화", "수", "목", "금", "토", "일" };
    }

    @GetMapping("test3")
    public Product test3() {
        return new Product("맥주", 2000);
    }

    @GetMapping("test4")
    public Product[] test4() {
        return new Product[] {
            new Product("맥주", 2000),
            new Product("우유", 1500)
        };
    }

}
```

▪ 컨트롤러 클래스

컨트롤러 클래스는 웹브라우저의 URL 요청을 받아서, 웹서버에서 실행되는 클래스이다.
웹브라우저가 웹서버에 어떤 URL을 요청을 하면, 그 URL에 해당하는 컨트롤러의 액션 메소드가 자동으로 호출되어 실행된다. 컨트롤러 클래스는 @RestController, @Controller 어노테이션 중의 하나가 붙어있어야 한다.

▪ 액션 메소드

웹브라우저가 어떤 URL을 웹서버에 요청하면, 그 요청된 URL에 해당하는 컨트롤러의 어떤 메소드가 자동으로 호출된다. 이렇게 웹브라우저의 요청에 의해서 자동으로 호출되는 컨트롤러의 메소드를 액션 메소드라고 부른다.

▪ 액션 메소드 URL

컨트롤러 클래스에 붙은 @RequestMapping("**first**") 어노테이션의 **first** 부분과
액션 메소드에 붙은 @GetMapping("**test1**") 어노테이션의 **test** 부분이 결합된 **first/test1** 이
그 액션 메소드의 URL 이다.

http://localhost:8088/first/test1 URL을 웹브라우저가 웹서버에 요청하면,
위 FirstController 컨트롤러 클래스의 test1 액션 메소드가 자동으로 호출되어 실행된다.

자동으로 호출할 액션 메소드를 찾을 때 컨트롤러 클래스 이름이나 액션 메소드의 이름은 상관 없고,
@RequestMapping, @GetMapping 어노테이션에 등록된 URL만 일치하면 된다.

@RestController

이 어노테이션이 붙은 컨트롤러를 레스트 컨트롤러라고 부른다.

레스트 컨트롤러 액션 메소드의 리턴값은 데이터이다.
그 데이터가 웹브라우저에 전송된다.
그 데이터는 JSON 형태의 텍스트로 변환되어 웹브라우저에 전송된다.

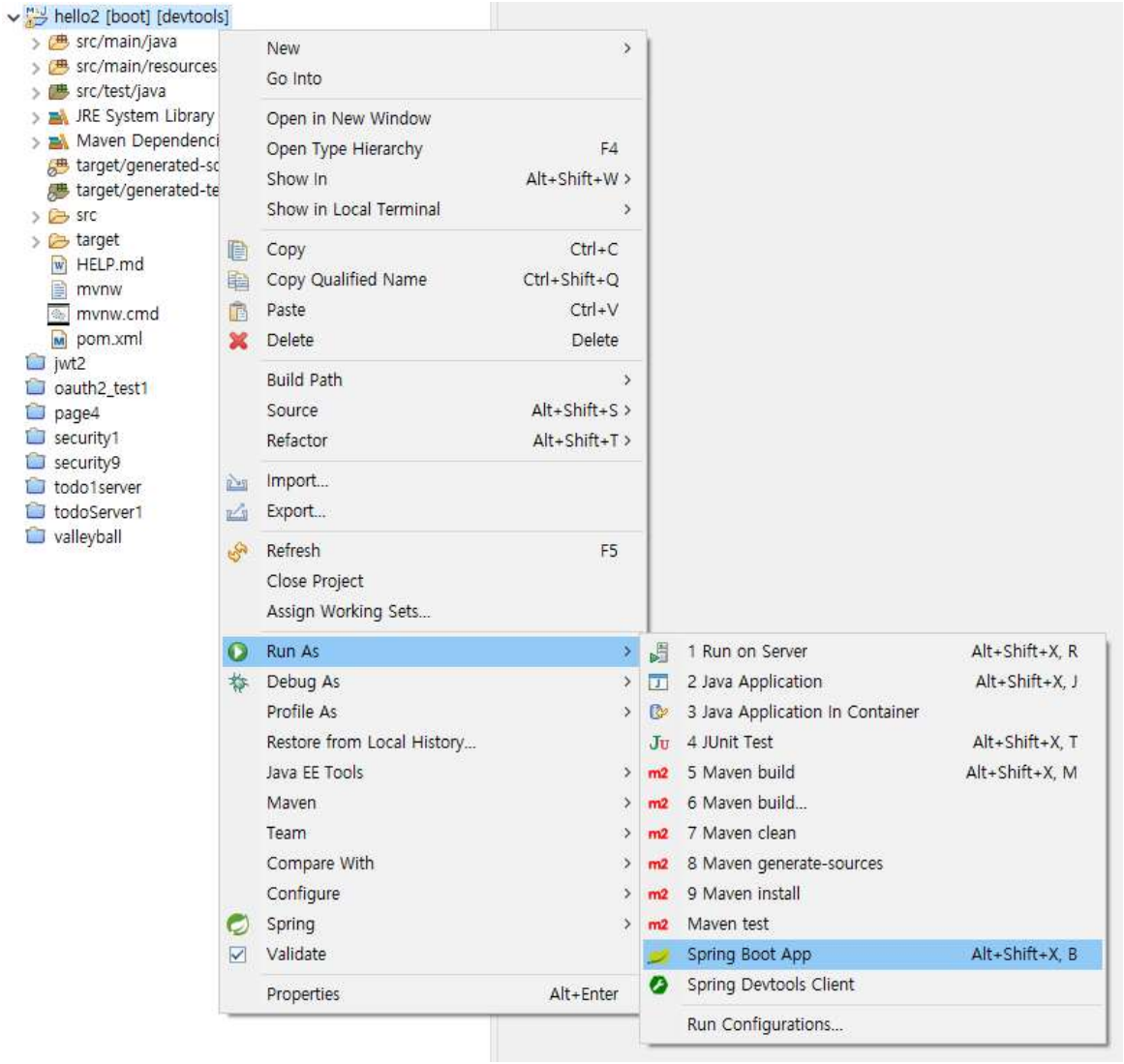
JSON (Javascript Object Notation)

@Controller

이 어노테이션이 붙은 컨트롤러는 그냥 컨트롤러라고 부른다.

이 컨트롤러의 액션메소드의 리턴값은 뷰(view) 템플릿 파일의 경로이다.
액션메소드가 리턴한 후, 그 경로의 뷰 파일이 실행된다.
뷰 파일의 소스코드가 실행되어 출력된 html 태그들이 웹브라우저에 전송된다.

4) 실행

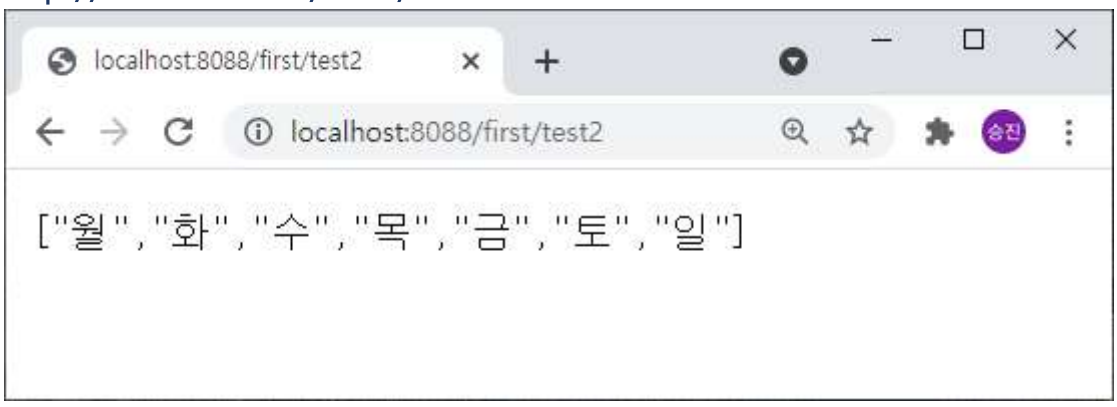


http://localhost:8088/first/test1



test1 액션 메소드가 리턴한 문자열이 그대로 웹브라우저에 전송되었다.

http://localhost:8088/first/test2



test2 액션 메소드가 리턴한 String 배열이 JSON 포멧으로 웹브라우저에 전송되었다.
웹브라우저에 보이는 내용이 Stirng 배열 JSON 포맷이다.

http://localhost:8088/first/test3

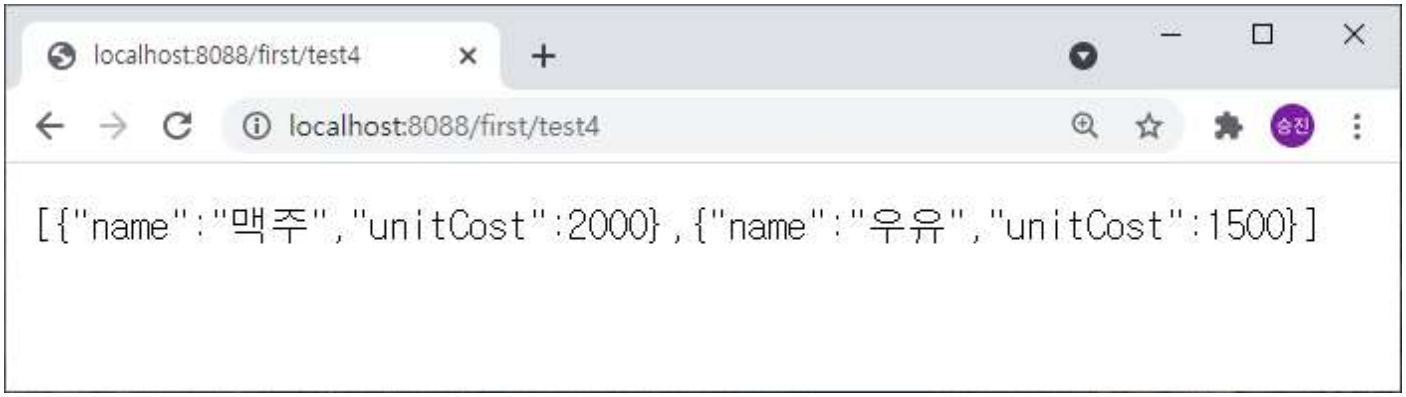


test3 액션 메소드가 리턴한 Product 객체가 JSON 포멧으로 웹브라우저에 전송되었다.
웹브라우저에 보이는 내용이 객체 한 개의 JSON 포맷이다.

여기서 "name", "unitCost" 부분은
Project Java 객체의 멤버 변수 이름이 아니고, 속성명이다. 즉 get 메소드 이름이다.

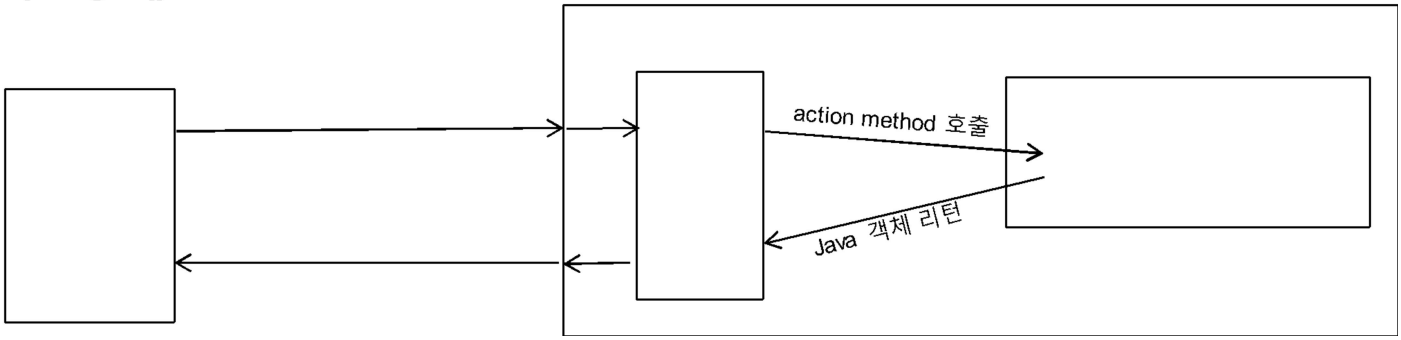
Java 객체를 JSON 형태의 객체로 변환할 때,
Java 객체 속성이 JSON 객체의 속성으로 변환된다.

http://localhost:8088/first/test4



test4 액션 메소드가 리턴한 Product 객체 배열이 JSON 포멧으로 웹브라우저에 전송되었다.
웹브라우저에 보이는 내용이 객체 배열의 JSON 포맷이다.

5) FirstController 액션 메소드 실행 절차



1	웹브라우저가 서버에 **요청(HTTP Request)**을 전달한다. 이 요청에는, 요청 대상을 가르키는 **URL**이 담겨 있다. (URL: http://localhost:8088/first/test3)
2	웹브라우저로부터 서버에 전달된 요청을 Spring Web MVC 엔진이 받는다. 스프링 엔진은 요청된 URL과 일치하는 **컨트롤러 액션 메소드**를 찾아서 호출한다. (FirstController 클래스의 test3() 액션 메소드 호출)
3	FirstController의 test3() 액션 메소드는 Java 객체를 리턴한다. (Product 객체 리턴)
4	리턴된 Java 객체는, Spring Web MVC 엔진에 의해서 JSON 형태의 문자열로 변경된다. {"name":"맥주","unitCost":2000, "quantity":120}
5	JSON 형태의 문자열이 웹브라우저에 전송된다. 이 전송은 최초 웹브라우저의 요청(http request)에 대한 응답(http response)이다.
6	웹 서버로부터 전송된 JSON 형태의 문자열이 웹브라우저에 표시된다.

6) JSON 형태의 텍스트

서버와 데이터를 주고 받을 때, 데이터를 JSON 형태의 텍스트로 변환해서 전송하는 경우가 흔하다.

JSON 문자열

```
"안녕하세요"
```

JSON 숫자 배열

```
[1, 2, 3, 4, 5]
```

JSON 문자열 배열

```
["월","화","수","목","금","토","일"]
```

JSON 객체 예#1

```
{"name":"맥주","unitCost":2000, "quantity":120}
```

JSON 객체 예#2

```
{"studentNumber":"201532045","name":"홍길동", "email":"hgd@skhu.ac.kr" }
```

JSON 객체 배열의 예#1

```
[{"name":"맥주","unitCost":2000},{"name":"우유","unitCost":1500}]
```

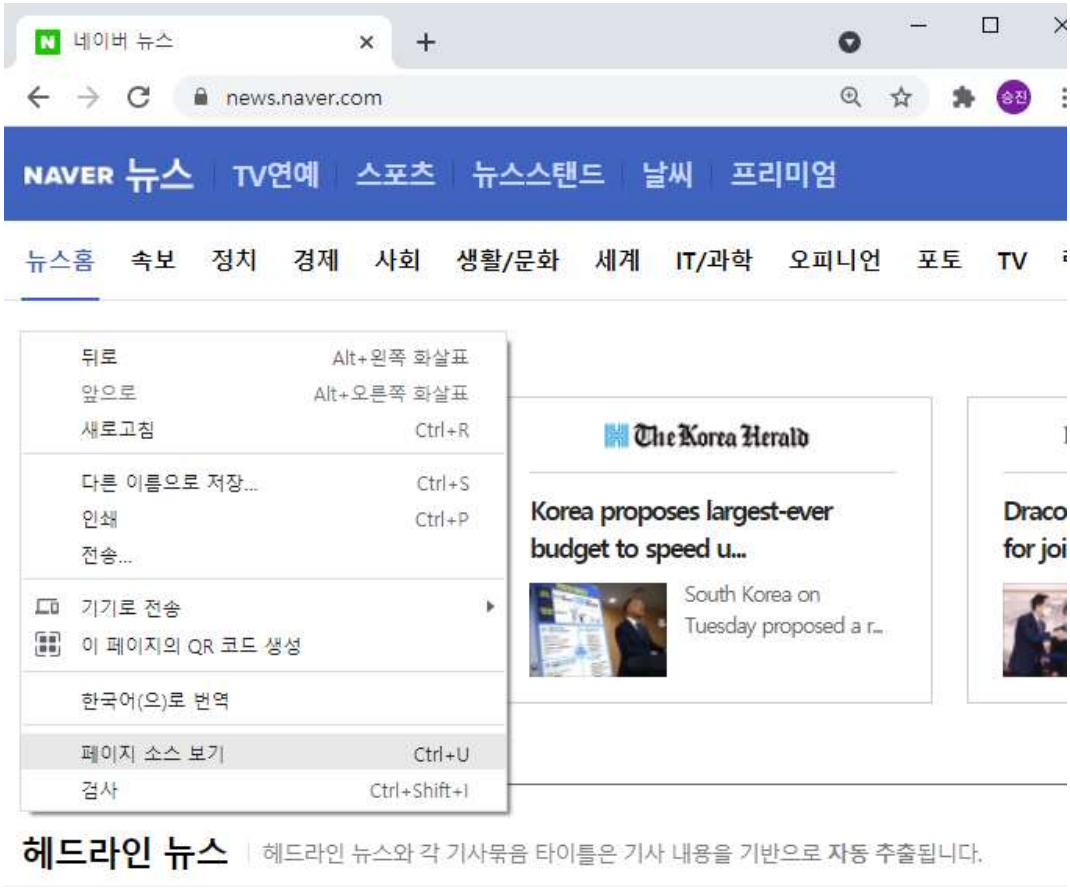
JSON 객체 배열의 예#2

```
[{"studentNumber":"201532045","name":"홍길동", "email":"hgd@skhu.ac.kr" },
 {"studentNumber":"201532046","name":"전우치", "email":"jwc@skhu.ac.kr" },
 {"studentNumber":"201532047","name":"임꺽정", "email":"ikj@skhu.ac.kr" }]
```

4. SecondController

1) 뷰(view)

웹브라우저의 요청(http request)에 대한 응답(http response)으로,
웹서버에서 웹브라우저로 전송되는 것은 대부분 html 태그이다.
우리가 웹서핑하면서 보는 웹페이지들의 내용이 html 태그이다.



웹페이지를 마우스 오른쪽 버튼으로 클릭하고, "페이지 소스 보기" 메뉴를 클릭하면
웹서버에서 웹브라우저로 전송된 내용인, html 태그를 볼 수 있다.

컨트롤러 액션 메소드 실행 결과로 웹브라우저로 전송되는 것이 html 태그인 경우에는
뷰(view) 파일을 구현해야 한다.

2) 실행 순서

웹브라우저에서 웹서버에 어떤 URL을 요청하면 (http request),
요청된 URL과 일치하는, 컨트롤러의 액션 메소드가 실행된다.

그리고 액션 메소드의 뒤를 이어서 뷰(view) 템플릿 파일이 실행된다.

뷰 템플릿 파일의 소스코드가 실행되어 출력된 html 태그들이 웹브라우저로 전송된다.

3) second/test1.html 뷰 파일 생성

src/main/resources/templates 폴더 아래에 second 폴더를 만들고, 그 아래에 test1.html 파일을 생성하자.

src/main/resources/templates/second/test1.html

```
1  <!DOCTYPE html>
2  <html lang="ko" xmlns:th="http://www.thymeleaf.org">
3  <head>
4    <meta charset="utf-8">
5  </head>
6  <body>
7    <h1 th:text="${message}">메시지가 없습니다</h1>
8  </body>
9  </html>
```

이 뷰 파일의 경로가 second/test1.html 이므로
액션 메소드는 다음과 같은 문자열을 리턴해야 한다.

```
return "second/test1";
```

```
<html lang="ko" xmlns:th="http://www.thymeleaf.org">
```

위 태그는 다음과 같이 선언한다.
이 html 문서의 언어는 한국어고 (ko)
이 html 문서에는 thymeleaf 확장 태그가 포함되어 있고, 그 확장 태그에는 th: 접두어가 붙어있다.
예: th:text="${message}"

```
<meta charset="utf-8">
```

이 문서의 문자 인코딩은 "utf-8" 이다.
한글이 깨지지 않고 제대로 보이려면, 인코딩 선언이 필요하다.

```
<h1 th:text="${message}">메시지가 없습니다</h1>
```

뷰에 전달된 model attribute 데이터 중에서, 이름이 "message"인 데이터가 이 태그 사이에 출력된다.
따라서 '메시지가 없습니다' 문구는 출력되지 않는다.

액션 메소드는 다음과 같이 "message" 이름의 model attribute를 전달해야 한다.

```
model.addAttribute("message", "안녕하세요");
```

위 뷰 소스코드가 출력하는 html 태그는 다음과 같다.

```
<h1>안녕하세요</h1>
```

4) second/test2.html 뷰 파일 생성

src/main/resources/templates/second/test2.html

```
1  <!DOCTYPE html>
2  <html lang="ko" xmlns:th="http://www.thymeleaf.org">
3  <head>
4    <meta charset="utf-8">
5  </head>
6  <body>
7    <h1 th:text="${product.name}">제품명</h1>
8    <h1 th:text="${product.unitCost}">1200</h1>
9  </body>
10 </html>
```

```
<h1 th:text="${product.name}">제품명</h1>
```

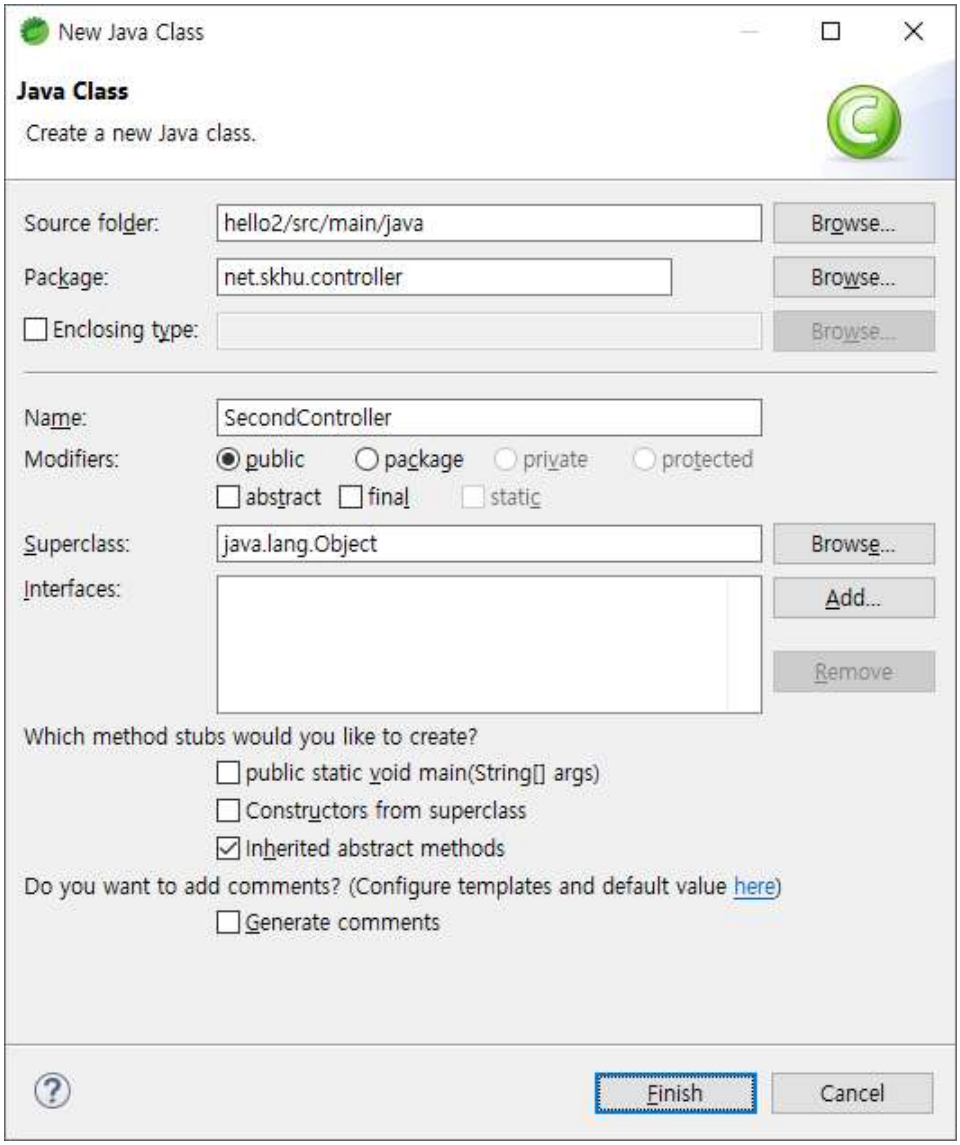
뷰에 전달된 데이터(model attribute) 중에서, 이름이 "product"인 객체의 name 속성값이 출력된다.
즉 getName() 메소드가 호출되고,
이 getName() 메소드가 리턴한 값이, 이 태그 사이에 출력된다.

액션 메소드는 다음과 같이 "product" 이름의 model attribute를 전달해야 한다.

```
model.addAttribute("product", new Product("맥주", 2000));
```

"product" => model attribute 데이터의 이름이고, 문자열이다.
new Product("맥주", 2000) => model attribute 데이터의 값이고, java 객체이다.

5) SecondController.java 생성



src/main/java/net/skhu/controller/SecondController.java

```java
package net.skhu.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import net.skhu.dto.Product;

@Controller
@RequestMapping("second")
public class SecondController {

    @GetMapping("test1")
    public String test1(Model model) {
        model.addAttribute("message", "안녕하세요");
        return "second/test1";
    }

    @GetMapping("test2")
    public String test2(Model model) {
        Product product = new Product("맥주", 2000);
        model.addAttribute("product", product);
        return "second/test2";
    }

}
```

@Controller

어노테이션(annotation)을 붙여주지 않으면 컨트롤러 클래스가 실행되지 않는다.
(404 Not Found 에러)

이 컨트롤러의 액션 메소드가 리턴하는 문자열은 뷰의 이름이다.
리턴된 이름의 뷰 파일이 실행된다.

Model 객체

```java
public String test1(Model model) {
```

액션 메소드가 뷰에 전달할 데이터를, 이 Model 객체에 넣어서 전달한다.
즉 Model 객체는, 데이터 전달 상자라고 보면 된다.
이렇게 전달되는 데이터를 model attribute 라고 한다.

```java
model.addAttribute("message", "안녕하세요");
```

model attribuet 데이터의 이름은 "message" 이고, 값은 "안녕하세요" 문자열이다.

```java
Product product = new Product("맥주", 2000);
model.addAttribute("product", product);
```

model attribuet 데이터의 이름은 "product" 문자열이고,
model attribuet 데이터의 값은 product 객체이다.

위 model attribute 데이터들이 아래의 태그 사이에 출력된다.

```html
<h1 th:text="${message}">메시지가 없습니다</h1>
```

```html
<h1 th:text="${product.name}">제품명</h1>
```

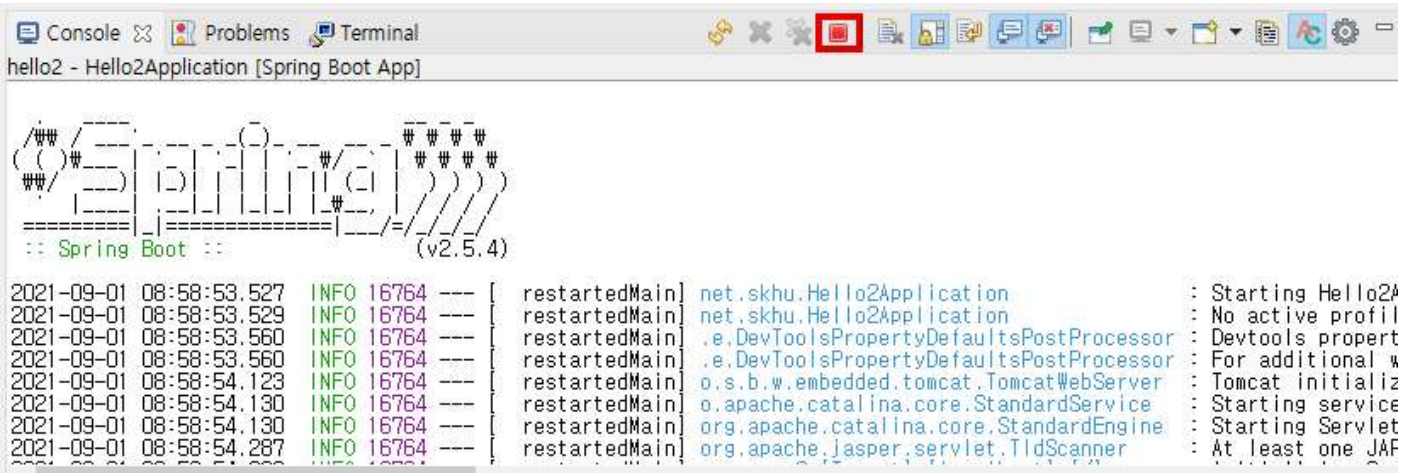
product.getName() 메소드가 호출되고 그 리턴값이 출력된다.

```html
<h1 th:text="${product.unitCost}">1200</h1>
```

product.getUnitCost() 메소드가 호출되고 그 리턴값이 출력된다.

6) 실행

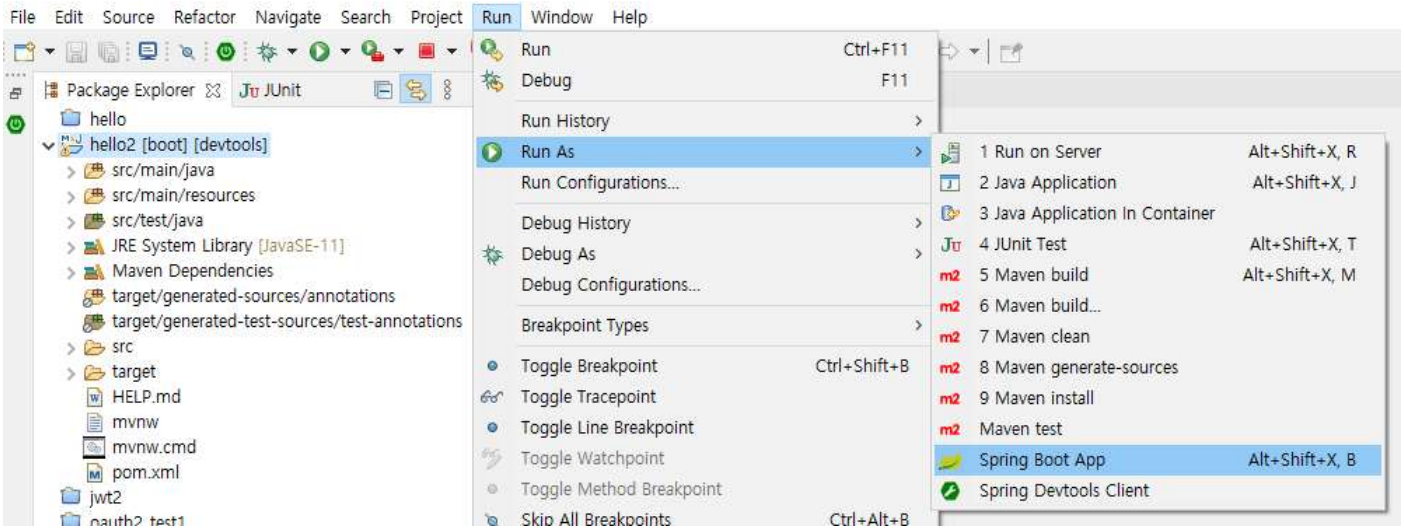
주의사항



만약 console 창의 내용이 위와 같다면, 이미 프로젝트가 8088 포트에서 실행 중이다.
(빨간색 버튼에 주목하자. 이 버튼을 클릭하면 프로젝트 실행이 종료된다)

이 상태에서는 Run 메뉴를 또 실행하면 안된다.
프로젝트를 또 실행되어 8088 포트 충돌이 발생할 것이다.

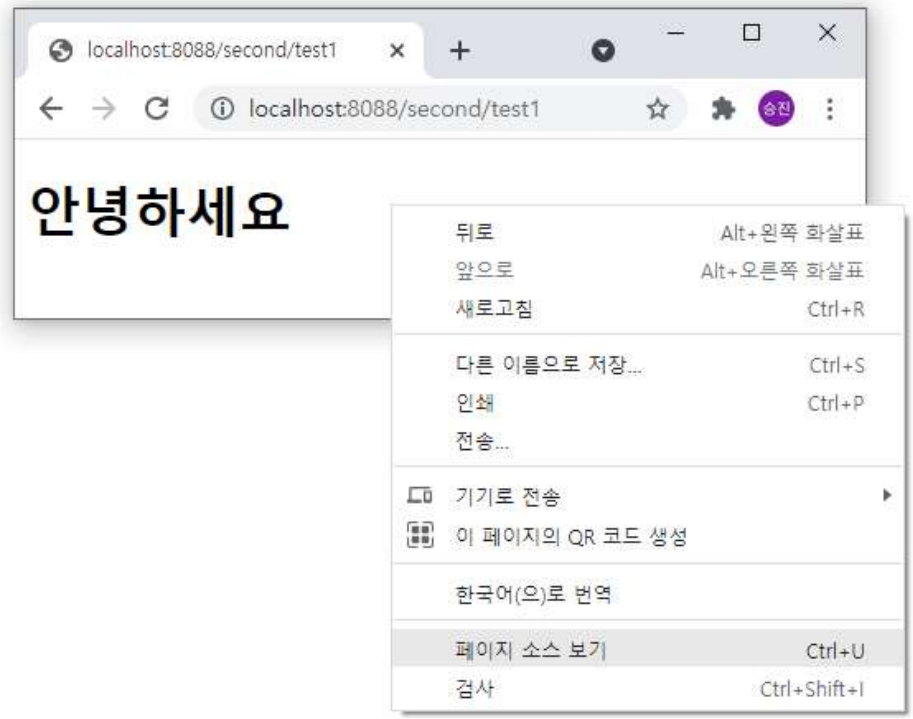
실행



http://localhost:8088/second/test1



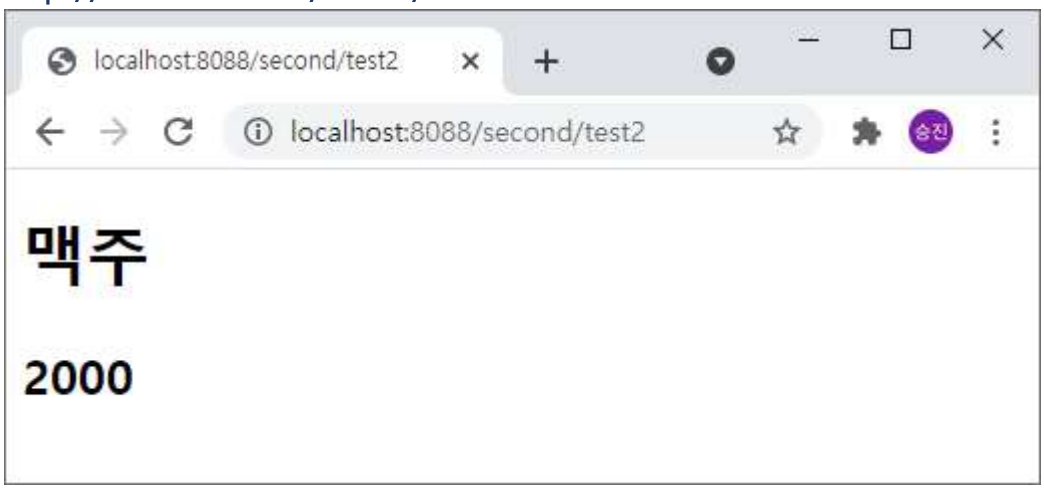
second/test1 실행화면 소스보기




```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="utf-8">
</head>
<body>
  <h1>안녕하세요</h1>
</body>
</html>
```

웹브라우저로 전달된 (http response) html 태그는 위와 같다.
thymeleaf 확장 태그가 어떻게 출력되었는지, test1.html 소스코드와 비교해 보자.

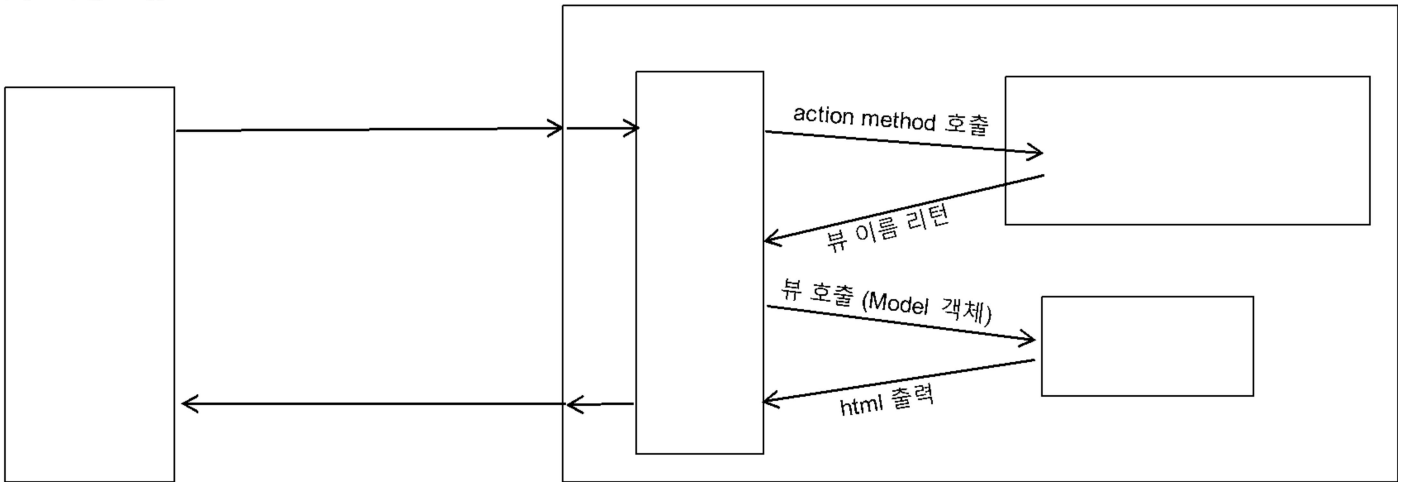
http://localhost:8088/second/test2




```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="utf-8">
</head>
<body>
  <h1>맥주</h1>
  <h1>2000</h1>
</body>
</html>
```

웹브라우저로 전달된 (http response) html 태그는 위와 같다.

7) 실행 절차



1	웹브라우저가 웹서버에 **요청(HTTP Request)**을 전달한다. 이 요청에는, 요청 대상을 가르키는 **URL**이 담겨 있다.
2	웹브라우저로부터 웹서버에 전달된 요청을 Spring Web MVC 엔진이 받는다. 스프링 엔진은 요청된 URL과 일치하는 **컨트롤러 액션 메소드**를 찾아서 호출한다.
3	컨트롤러의 액션 메소드는 데이터를 Model 객체에 넣는다.
4	컨트롤러의 액션 메소드는 뷰의 이름을 리턴한다.
5	그 이름의 뷰가 실행된다.
6	뷰는 Model 객체에서 데이터를 꺼내서 출력하고, HTML 태그들도 출력한다.
7	뷰가 출력한 HTML 태그들이 웹브라우저에 전송된다. 이 전송은 최초 웹브라우저의 요청(http request)에 대한 응답(http response)이다.
8	웹 서버로부터 전송된 HTML 태그들이 웹브라우저에 표시된다.

5. HomeController

1) HomeController.java 생성

src/main/java/net/skhu/controller/HomeController.java

```
1  package net.skhu.controller;
2
3  import org.springframework.stereotype.Controller;
4  import org.springframework.web.bind.annotation.RequestMapping;
5
6  @Controller
7  public class HomeController {
8
9      @GetMapping("/")
10     public String index() {
11         return "home/index";
12     }
13
14 }
```

액션 메소드

@GetMapping("/") 어노테이션의 "/" 부분은
그 액션 메소드를 호출하기 위한 URL 이다.

return "home/index";

액션 메소드가 리턴한 문자열은 "home/index" 이다.
  액션 메소드가 리턴한 후
  src/main/resources/templates/home/index.html 뷰 템플릿 파일이 실행된다.

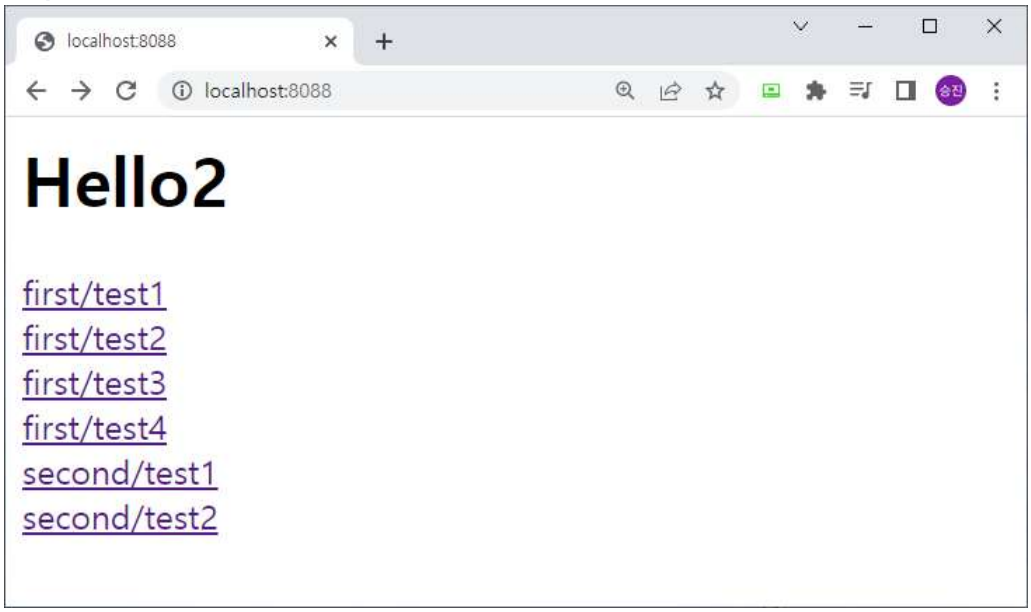
2) home/index.html 생성

src/main/resource/templates/home/index.html

```
1  <!DOCTYPE html>
2  <html lang="ko" xmlns:th="http://www.thymeleaf.org">
3  <head>
4    <meta charset="utf-8">
5  </head>
6  <body>
7  <h1>Hello2</h1>
8
9  <a href="first/test1">first/test1</a> <br />
10 <a href="first/test2">first/test2</a> <br />
11 <a href="first/test3">first/test3</a> <br />
12 <a href="first/test4">first/test4</a> <br />
13
14 <a href="second/test1">second/test1</a> <br />
15 <a href="second/test2">second/test2</a> <br />
16
17 </body>
18 </html>
```

3) 실행

http://localhost:8088



http://localhost:8088/first/test3 클릭



위 실행 절차 설명

1	hello2 프로젝트를 실행하면, hello2 프로젝트가 빌드되고, 빌드된 배포 파일(hello2.war) 내부의 톰켓 서버가 실행된다.
2	웹브라우저가 **http://localhost:8088/** URL을 요청하면, 서버에서 HomeController의 index 메소드가 호출된다.
3	src/main/resource/templates/index.html 파일이 실행되고, 그 실행결과 출력이 웹브라우저에 전송된다.
4	전송된 html 태그가 웹브라우저에 표시된다.
5	사용자가 웹브라우저에서 first/test3 링크를 마우스로 클릭한다.
6	웹브라우저가 **http://locahost:8088/first/test3** URL을 웹서버에 요청한다.
7	FirstController의 test3 액션 메소드가 서버에서 실행된다.
8	test3 액션 메소드가 리턴한 데이터가 웹브라우저에 전송된다.
9	전송된 데이터가 웹브라우저에 표시된다.

6. 구현 과제

1) 프로젝트 생성

프로젝트명: hwA학번
예: hwA200814199

프로젝트 새로 생성하지 않고, hello2 프로젝트에 구현해도 됨.

2) Student 클래스 생성

멤버 변수

```
int id;
String studentNumber;
String studentName;
String email;
```

3) ThirdController 클래스 생성

다음과 같은 절차의 test1 액션 메소드 구현

1. Student 객체를 생성한다.
2. Student 객체에 id, studentNumber, studentName, email 값을 대입
   id 변수에는 아무 값이나 채우고, 나머지 멤버 변수에는 자신의 학번,이름, 이메일 주소를 채운다.
3. Model 객체에 student 객체를 넣는다.
4. 뷰 템플릿 파일의 경로를 리턴

4) third/test1.html 뷰 템플릿 파일 생성

전달된 student 객체의 id, studnetNumber, studentName, email 값을
<table> 태그를 사용해서 출력하라.

실행 URL

test1 액션 메소드를 호출하기 위한 URL은 다음과 같아야 한다.
http://localhost:8088/third/test1

제출할 내용

① 프로젝트 디렉토리 zip 압축
   엉뚱한 디렉토리를 압축해서 제출하면 감점
   압축파일 크기가 1M를 넘으면 엉뚱한 디렉토리를 압축한 것임

② 실행 웹브라우저 창 캡처 이미지 파일

7. 요약

@Controller 컨트롤러

액션 메소드가 리턴하는 값은 뷰 템플릿 파일의 경로
  액션 메소드가 리턴한 후 그 뷰 템플릿 파일이 실행되고
  뷰 템플릿 파일의 소스코드가 실행되어 출력된 html 태그들이 웹브라우저에 전송된다.

@RestController 컨트롤러

액션 메소드가 리턴하는 값은 데이터 (Java 객체나 배열)
  리턴된 데이터가 JSON 포맷의 문자열로 자동 변환되어
  웹브라우저로(프론트엔드로) 전송된다.